ONE-PAGE BRIEF

Post-quantum cryptography review: safe-smart-account @ 247b8ce3bdbf

Client: Safe — public repository (passive review) · Source: github.com/safe-global/safe-smart-account, commit `247b8ce3bdbf` · Reviewed: 2026-11-06 13:53 UTC · Risk profile: Crypto exchange, custodian or wallet

POST-QUANTUM MATURITY
Level 0 of 4
no post-quantum cryptography

FINDINGS
0 Critical **0** High **3** Medium
3 Low **2** Info

QUANTUM TIMELINE (MOSCA: X + Y − Z)
Signatures: 10 + 5 − 8 = 7 → **too late**

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

WHAT MATTERS MOST

Medium QSI-2026-18794	**Raw ecrecover usage in certora/harnesses/SafeHarness.sol** preliminary What to do: Use OpenZeppelin ECDSA.recover with EIP-712 domain separation and nonces.
Medium QSI-2026-18795	**Unchecked low-level call in certora/harnesses/SafeHarness.sol** preliminary What to do: Check the boolean return value or use OpenZeppelin Address.sendValue / functionCall.
Medium QSI-2026-18793	**Quantum-vulnerable ECDSA: transaction and message signing (3 file(s))** preliminary What to do: Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains...
Low QSI-2026-18796	**Floating pragma (>=0.7.0 <0.9.0) in 50 file(s)** What to do: Pin the compiler version used in testing.

2 more in the full report.

WHAT THE QUANTUM-VULNERABLE CRYPTOGRAPHY PROTECTS

Role	Algorithms	What to do
Medium transaction and message signing	ECDSA	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum...

Static review of source code and manifests; files analysed: 77. No requests were sent to the owner's systems.

Full report, content SHA-256: `a78c7f23d1423e7637d8911f87319e67fed03020af9361b43d60c729eb96d13b`

Full report signature (report.sig.json): Ed25519 key, SHA-256 fingerprint `5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`. bundle.sig.json lists the SHA-256 of every file in this bundle and is signed too; verify-bundle checks it offline.

Preliminary findings need confirmation in the code; the full report gives the evidence, the limits of each check and how to re-test.